

CleanAVLVisualizer.java - Detailed Explanation with Next/Prev Functionality

1. Code Snippet: AVLTree class update for Next/Prev buttons

```
// Inside AVLTree class
private List<Node> traversallist = new ArrayList<>();
private int currentIndex = -1;

// Generate in-order traversal
void generateTraversallist(){
    traversallist.clear();
    inOrder(root);
    currentIndex = -1;
}

private void inOrder(Node node){
    if(node == null) return;
    inOrder(node.left);
    traversallist.add(node);
    inOrder(node.right);
}

Node getNext(){
    if(traversallist.isEmpty()) return null;
    if(currentIndex < traversallist.size() - 1) currentIndex++;
    return traversallist.get(currentIndex);
}

Node getPrev(){
    if(traversallist.isEmpty()) return null;
    if(currentIndex > 0) currentIndex--;
    return traversallist.get(currentIndex);
}
```

Explanation: - `traversallist` stores nodes in in-order sequence for easy navigation. - `currentIndex` tracks the currently highlighted node. - `generateTraversallist()` fills the list by performing an in-order traversal. - `getNext()` moves forward in the list and returns the next node. - `getPrev()` moves backward and returns the previous node. - This is core logic that allows Next/Prev navigation independent of tree structure.

2. Code Snippet: Adding Next/Prev buttons in the Control Panel

```
JButton nextBtn = new JButton("Next");
JButton prevBtn = new JButton("Prev");
styleButton(nextBtn, new Color(255, 152, 0));
styleButton(prevBtn, new Color(255, 87, 34));

control.add(nextBtn);
control.add(prevBtn);

nextBtn.addActionListener(e -> {
    avl.generateTraversallList();
    Node next = avl.getNext();
    if(next != null){
        next.popping = true;
        drawPanel.highlightNode(next);
    }
});

prevBtn.addActionListener(e -> {
    avl.generateTraversallList();
    Node prev = avl.getPrev();
    if(prev != null){
        prev.popping = true;
        drawPanel.highlightNode(prev);
    }
});
```

Explanation: - Buttons are created and styled using `styleButton`. - Added to the existing control panel at the top. - ActionListeners call `generateTraversallList()` to update traversal order. - `highlightNode()` is called on the drawing panel to visually emphasize the selected node. - Ensures synchronization with the in-order sequence of the AVL tree.

3. Code Snippet: Highlighting Node in DrawPanel

```
// Inside DrawPanel class
private Node highlightedNode = null;

void highlightNode(Node node){
    highlightedNode = node;
    repaint();
}

@Override
protected void paintComponent(Graphics g){
```

```

    super.paintComponent(g);
    setBackground(Color.WHITE);
    if(avl.root != null)
        drawNode((Graphics2D)g, avl.root, getWidth()/2, 60, getWidth()/4);
}

void drawNode(Graphics2D g2, Node node, int x, int y, int xOffset){
    // Draw edges
    if(node.left != null) drawEdge(g2, x, y, x-xOffset, y+levelHeight,
node.left);
    if(node.right != null) drawEdge(g2, x, y, x+xOffset, y+levelHeight,
node.right);

    // Highlight logic
    if(node == highlightedNode){
        g2.setColor(Color.YELLOW);
        g2.fillOval(x-nodeSize/2-5, y-nodeSize/2-5, nodeSize+10, nodeSize+10);
    }

    // Node drawing as usual
    g2.setColor(node.toRemove ? new Color(255, 100, 100,
Math.round(node.alpha*255)) : new Color(0,120,255));
    g2.fillOval(x-nodeSize/2, y-nodeSize/2, nodeSize, nodeSize);
    g2.setColor(Color.BLACK);
    g2.drawOval(x-nodeSize/2, y-nodeSize/2, nodeSize, nodeSize);

    // Draw key and balance factor
    drawKeyAndBF(g2, node, x, y);

    if(node.left != null) drawNode(g2,node.left,x-xOffset,y+levelHeight,xOffset/
2);
    if(node.right != null)
drawNode(g2,node.right,x+xOffset,y+levelHeight,xOffset/2);
}

```

Explanation: - `highlightNode` sets which node to emphasize. - In `drawNode`, a yellow highlight is drawn behind the selected node. - This works with Next/Prev buttons to visually navigate through the tree.

4. Layout and Swing Components

- `JFrame` is the main window.
- `BorderLayout` divides frame into `CENTER` (`DrawPanel`), `NORTH` (control buttons), `SOUTH` (explanation `JTextArea`).
- `JPanel` for controls holds `JTextField`s for insert/delete and buttons.
- `JTextArea` inside `JScrollPane` shows rotation explanations.
- `JBUTTON`s are styled with hover effects using `MouseAdapter`.

- `DrawPanel` extends `JPanel` and overrides `paintComponent` for custom graphics.
-

5. Balance Factor Implementation

```
int balance(Node n) { return n == null ? 0 : height(n.left) - height(n.right); }
```

- Calculated as height difference between left and right subtrees. - Shown above nodes in `drawNode` using `Graphics2D`. - Helps visualize AVL rebalancing operations. - Updated after every insert/delete with `updateHeight()`.

6. Summary of Next/Prev Functionality

- `generateTraversallist` ensures the current tree structure is reflected.
- `getNext` and `getPrev` handle in-order traversal navigation.
- `highlightNode` visually marks the current node.
- Buttons are integrated into existing control panel.
- Fully interacts with existing animated insert/delete logic and rotation explanations.